

H524 Introduction to Biostatistics

Week 3 Lab: Probability Distributions and Sampling

Fall 2025

Format: Online

Tools Required: R, RStudio

Estimated Time: 90 minutes

1 Learning Objectives

By the end of this lab, you will be able to:

1. Work with the binomial distribution in R (`dbinom`, `pbinom`)
2. Calculate probabilities using the normal distribution (`dnorm`, `pnorm`, `qnorm`)
3. Understand standardization (Z-scores) and use R to convert between scales
4. Simulate sampling distributions to demonstrate the Central Limit Theorem
5. Calculate and interpret confidence intervals in R
6. Visualize probability distributions and sampling distributions
7. [Use AI tools to help with distribution problems and R code](#)

2 Part 1: Binomial Distribution in R

The binomial distribution models the number of successes in a fixed number of independent trials, each with the same probability of success.

2.1 Exercise 1.1: Basic Binomial Calculations

Scenario: A vaccine has 85% efficacy. We vaccinate 20 people.

```
# Binomial distribution: n=20, p=0.85
n <- 20
p <- 0.85

# Probability of exactly 17 protected
prob_17 <- dbinom(17, size=n, prob=p)
cat("P(X = 17):", round(prob_17, 4), "\n")

# Probability of 17 or fewer protected
prob_at_most_17 <- pbinom(17, n, p)
cat("P(X <= 17):", round(prob_at_most_17, 4), "\n")

# Probability of at least 18 protected
prob_at_least_18 <- pbinom(17, n, p, lower.tail=FALSE)
# OR: 1 - pbinom(17, n, p)
```

```
cat("P(X >= 18):", round(prob_at_least_18, 4), "\n")

# Expected value and standard deviation
expected <- n * p
std_dev <- sqrt(n * p * (1 - p))
cat("\nExpected value:", expected, "\n")
cat("Standard deviation:", round(std_dev, 3), "\n")
```

Try it yourself: What if the efficacy was 90% instead? Recalculate all the above probabilities with $p = 0.90$.

2.2 Exercise 1.2: Visualizing the Binomial Distribution

```
# Create a visualization of the binomial distribution
n <- 20
p <- 0.85

# Create vector of possible values
x_values <- 0:n

# Calculate probabilities for each value
probabilities <- dbinom(x_values, n, p)

# Create barplot and capture bar positions
bp <- barplot(probabilities,
              names.arg = x_values,
              main = "Binomial Distribution (n=20, p=0.85)",
              xlab = "Number of Successes",
              ylab = "Probability",
              col = "lightblue",
              border = "white")

# Add vertical line at expected value
abline(v = bp[which(x_values == n*p)], col = "red", lwd = 2, lty = 2)

# Add legend
legend("topleft",
      legend = "Expected Value",
      col = "red",
      lty = 2,
      lwd = 2)
```

Question: Look at your plot. Does it appear symmetric or skewed? Why?

2.3 Exercise 1.3: Binomial Scenarios

```
# Antibiotic treatment: 75% success rate, treat 15 patients
antibiotic_prob <- dbinom(12, size=15, prob=0.75)
cat("P(exactly 12 successes):", round(antibiotic_prob, 4), "\n")

# At least 10 successes
at_least_10 <- sum(dbinom(10:15, size=15, prob=0.75))
# OR: pbinom(9, 15, 0.75, lower.tail=FALSE)
cat("P(at least 10 successes):", round(at_least_10, 4), "\n")
```

Practice: A clinical trial tests a drug on 25 patients. The success rate is 0.70. Find:

- a) $P(\text{exactly 18 successes})$
- b) $P(\text{fewer than 15 successes})$
- c) $P(\text{between 15 and 20 successes, inclusive})$

3 Part 2: Normal Distribution in R

The normal distribution is the most important continuous distribution in statistics. R provides four functions: `dnorm` (density), `pnorm` (cumulative probability), `qnorm` (quantiles), and `rnorm` (random generation).

3.1 Exercise 2.1: Basic Normal Calculations

Scenario: Cholesterol levels are normally distributed with mean 200 mg/dL and SD 40 mg/dL.

```
# Normal distribution parameters
mu <- 200      # mean
sigma <- 40    # standard deviation

# Probability that cholesterol < 240
prob_below_240 <- pnorm(240, mean=mu, sd=sigma)
cat("P(X < 240):", round(prob_below_240, 4), "\n")

# Probability that cholesterol > 240
prob_above_240 <- pnorm(240, mu, sigma, lower.tail=FALSE)
# OR: 1 - pnorm(240, mu, sigma)
cat("P(X > 240):", round(prob_above_240, 4), "\n")

# Probability between 180 and 220
prob_between <- pnorm(220, mu, sigma) - pnorm(180, mu, sigma)
cat("P(180 < X < 220):", round(prob_between, 4), "\n")
```

Question: About what proportion of people have cholesterol above 240 mg/dL (considered high)?

3.2 Exercise 2.2: Finding Percentiles

```
# Find the 90th percentile of cholesterol
percentile_90 <- qnorm(0.90, mean=mu, sd=sigma)
cat("90th percentile:", round(percentile_90, 1), "mg/dL\n")

# Find the 25th, 50th (median), and 75th percentiles
quartiles <- qnorm(c(0.25, 0.50, 0.75), mu, sigma)
names(quartiles) <- c("Q1", "Median", "Q3")
print(round(quartiles, 1))

# Verify: what proportion is below the 90th percentile?
check <- pnorm(percentile_90, mu, sigma)
cat("\nVerification:", round(check, 2), "\n")
```

Try it yourself: Find the cholesterol levels that define:

- The bottom 10% (10th percentile)
- The top 5% (95th percentile)

3.3 Exercise 2.3: Standardization (Z-scores)

```
# Convert a cholesterol value to a Z-score
chol_value <- 260 # mg/dL

# Calculate Z-score manually
z_score <- (chol_value - mu) / sigma
cat("Z-score for", chol_value, "mg/dL:", round(z_score, 2), "\n")

# Interpretation
cat("This value is", round(z_score, 2),
    "standard deviations above the mean\n")

# Use the standard normal to find probability
prob <- pnorm(z_score)
cat("Proportion below", chol_value, ":", round(prob, 4), "\n")

# Alternative: use original scale directly
prob_direct <- pnorm(chol_value, mu, sigma)
cat("Direct calculation:", round(prob_direct, 4), "\n")
```

```
# R also has a scale() function for vectors
cholesterol_data <- c(180, 220, 195, 240, 175, 210)
z_scores <- scale(cholesterol_data, center=mu, scale=sigma)
print(round(z_scores, 2))
```

Question: If someone has a Z-score of 1.5, what is their actual cholesterol level?

3.4 Exercise 2.4: Visualizing the Normal Distribution

```
# Create a normal distribution plot
x <- seq(100, 300, length=200)
y <- dnorm(x, mean=mu, sd=sigma)

plot(x, y, type="l", lwd=2, col="blue",
     main="Normal Distribution of Cholesterol",
     xlab="Cholesterol (mg/dL)",
     ylab="Density")

# Add vertical lines for mean and +/- 1 SD
abline(v=mu, col="red", lwd=2, lty=1)
abline(v=c(mu-sigma, mu+sigma), col="red", lwd=1, lty=2)

# Shade region above 240
x_high <- seq(240, 300, length=100)
y_high <- dnorm(x_high, mu, sigma)
polygon(c(240, x_high, 300), c(0, y_high, 0),
       col=rgb(1,0,0,0.3), border=NA)

# Add text
text(mu, max(y)*0.9, "Mean", pos=3)
text(240, max(y)*0.5, "High\nCholesterol", pos=4)

# Add legend
legend("topright",
     legend=c("Mean", "+/- 1 SD"),
     col="red",
     lty=c(1,2),
     lwd=c(2,1))
```

4 Part 3: Sampling Distributions and Central Limit Theorem

Now we'll see one of the most important concepts in statistics: the Central Limit Theorem (CLT).

4.1 Exercise 3.1: Understanding Standard Error

```
# Population parameters
pop_mean <- 120      # mean blood pressure
pop_sd <- 20         # population SD

# Calculate standard error for different sample sizes
sample_sizes <- c(5, 10, 25, 50, 100)

for (n in sample_sizes) {
  se <- pop_sd / sqrt(n)
  cat("n =", n, "\t SE =", round(se, 2), "\n")
}

# Notice: SE decreases as n increases!
```

Question: To cut the standard error in half, by what factor must you increase the sample size?

4.2 Exercise 3.2: Simulating the Sampling Distribution

```
# Simulate drawing many samples and computing their means
set.seed(524)

pop_mean <- 120
pop_sd <- 20
sample_size <- 30
num_samples <- 1000

# Draw 1000 samples, each of size 30, and compute their means
sample_means <- replicate(num_samples, {
  sample_data <- rnorm(sample_size, mean=pop_mean, sd=pop_sd)
  mean(sample_data)
})

# Summary of the sampling distribution
cat("Mean of sample means:", round(mean(sample_means), 2), "\n")
cat("SD of sample means (empirical SE):", round(sd(sample_means), 2), "\n")
cat("Theoretical SE:", round(pop_sd/sqrt(sample_size), 2), "\n")

# Visualize the sampling distribution
hist(sample_means, breaks=30, prob=TRUE,
      main=paste("Sampling Distribution of Mean (n=", sample_size, ")", sep=""),
      xlab="Sample Mean",
      ylab="Density",
      col="lightblue",
      border="white")
```

```
# Overlay theoretical normal distribution
se_theoretical <- pop_sd / sqrt(sample_size)
curve(dnorm(x, mean=pop_mean, sd=se_theoretical),
      add=TRUE, col="red", lwd=2)

legend("topright",
      legend="Theoretical (CLT)",
      col="red",
      lwd=2)
```

Observation: The distribution of sample means is approximately normal, even though we're sampling from a normal population. The CLT guarantees this!

4.3 Exercise 3.3: CLT with Non-Normal Data (Exponential Population)

Let's see the power of the CLT: it works even when the population is NOT normal!

Step 1: Define a population that is exponential (clearly NOT normal)

```
# Define our population: exponential distribution
# This represents waiting times, highly right-skewed
set.seed(123)

# Population parameters
pop_mean <- 50          # mean of exponential
rate_param <- 1/50      # rate parameter for rexp()

# Show what the population looks like
population <- rexp(10000, rate=rate_param)

# Plot the population distribution
hist(population, breaks=50, prob=TRUE,
     main="Population Distribution (Exponential)",
     xlab="Value",
     col="lightcoral",
     border="white")

# Add population density curve
curve(dexp(x, rate=rate_param), add=TRUE, col="darkred", lwd=2)

# Add statistics
text(100, 0.015, paste("Mean =", round(mean(population), 1)), pos=4)
text(100, 0.013, paste("SD =", round(sd(population), 1)), pos=4)
text(100, 0.011, "Clearly SKEWED!", pos=4, font=2)
```

Observation: The population is highly right-skewed (NOT normal at all!)

Step 2: Repeatedly sample from this population and calculate means

```
# Now let's see what happens when we take samples and compute means
```

```

set.seed(123)

# We'll try different sample sizes
sample_sizes <- c(5, 10, 30, 100)

# Compare sampling distributions for different sample sizes
par(mfrow=c(2,2)) # 2x2 plot layout

for (n in sample_sizes) {
  # Take 1000 samples of size n
  # For each sample, calculate the mean
  sample_means <- replicate(1000, {
    sample_data <- sample(population, n, replace=TRUE)
    mean(sample_data)
  })

  # Plot the distribution of sample means
  hist(sample_means, breaks=30, prob=TRUE,
       main=paste("Sample size n =", n),
       xlab="Sample Mean",
       col="lightblue",
       border="white",
       xlim=c(0, 100))

  # Overlay the theoretical normal distribution (CLT prediction)
  se_theoretical <- pop_mean / sqrt(n) # SE for exponential
  curve(dnorm(x, mean=pop_mean, sd=se_theoretical),
        add=TRUE, col="red", lwd=2)

  # Add text
  text(70, max(hist(sample_means, plot=FALSE)$density) * 0.8,
       paste("SE =", round(se_theoretical, 2)),
       col="red")
}

par(mfrow=c(1,1)) # Reset plot layout

```

Step 3: Observe that sample means ARE normally distributed!

Amazing CLT result:

- **Population:** Highly skewed exponential (definitely NOT normal)
- **Sample means:** Approximately normal distribution!
- **As n increases:** The approximation gets better and better
- Even at $n = 30$: Already looks quite normal
- At $n = 100$: Almost perfectly normal!

Key Insight: The CLT tells us that *no matter what the population distribution looks like*, sample means will be approximately normally distributed for large enough n . This is why the normal distribution is so fundamental to statistical inference!

4.4 Exercise 3.4: Probability Calculations with Sample Means

```
# Population: cholesterol with mean=200, SD=40
# Sample size: n=64

pop_mean <- 200
pop_sd <- 40
n <- 64

# Standard error
se <- pop_sd / sqrt(n)
cat("Standard Error:", se, "\n")

# Probability that sample mean > 210
prob_above_210 <- pnorm(210, mean=pop_mean, sd=se, lower.tail=FALSE)
cat("P(sample mean > 210):", round(prob_above_210, 4), "\n")

# Probability that sample mean is between 195 and 205
prob_between <- pnorm(205, pop_mean, se) - pnorm(195, pop_mean, se)
cat("P(195 < sample mean < 205):", round(prob_between, 4), "\n")

# Compare with individual probability
prob_individual <- pnorm(205, pop_mean, pop_sd) - pnorm(195, pop_mean, pop_sd)
cat("\nFor comparison, P(195 < individual < 205):",
    round(prob_individual, 4), "\n")
cat("Sample means are MUCH more concentrated around the population mean!\n")
```

5 Part 4: Confidence Intervals in R

Confidence intervals quantify our uncertainty about population parameters.

5.1 Exercise 4.1: Manual CI Calculation

```
# Sample data: tumor growth measurements (mm)
tumor_growth <- c(7, 10, 9, 8, 7, 6, 8, 9, 12, 13)

# Calculate sample statistics
xbar <- mean(tumor_growth)
s <- sd(tumor_growth)
n <- length(tumor_growth)

cat("Sample mean:", round(xbar, 2), "\n")
cat("Sample SD:", round(s, 3), "\n")
cat("Sample size:", n, "\n\n")

# Calculate standard error
se <- s / sqrt(n)
cat("Standard error:", round(se, 3), "\n\n")

# 95% Confidence Interval using t-distribution
```

```

alpha <- 0.05
df <- n - 1
t_critical <- qt(1 - alpha/2, df)

cat("Degrees of freedom:", df, "\n")
cat("t critical value:", round(t_critical, 3), "\n\n")

# Margin of error
margin_error <- t_critical * se

# Confidence interval
ci_lower <- xbar - margin_error
ci_upper <- xbar + margin_error

cat("95% Confidence Interval:\n")
cat("(", round(ci_lower, 2), ", ", round(ci_upper, 2), ")\n", sep="")

```

Interpretation: We are 95% confident that the true mean tumor growth is between 7.30 and 10.50 mm.

5.2 Exercise 4.2: Using t.test() for CIs

```

# Much easier method: use t.test()
result <- t.test(tumor_growth, conf.level=0.95)

# Extract confidence interval
ci <- result$conf.int
cat("95% CI from t.test():", round(ci, 2), "\n")

# The t.test() function gives us additional info
print(result)

# For 99% CI, just change conf.level
result_99 <- t.test(tumor_growth, conf.level=0.99)
cat("\n99% CI:", round(result_99$conf.int, 2), "\n")

```

Note: The `t.test()` function is the easiest way to get confidence intervals in R!

5.3 Exercise 4.3: Comparing Different Confidence Levels

```

# Compare 90%, 95%, and 99% confidence intervals
confidence_levels <- c(0.90, 0.95, 0.99)

for (conf in confidence_levels) {
  ci <- t.test(tumor_growth, conf.level=conf)$conf.int
  width <- ci[2] - ci[1]

  cat(conf*100, "% CI: (", round(ci[1], 2), ", ", round(ci[2], 2),
      ") Width:", round(width, 2), "\n", sep="")
}

```

```
}

cat("\nNotice: Higher confidence = wider interval\n")
```

5.4 Exercise 4.4: Effect of Sample Size on CIs

```
# Generate larger sample from same population
set.seed(42)

# Small sample (n=10)
small_sample <- rnorm(10, mean=100, sd=15)
ci_small <- t.test(small_sample)$conf.int
width_small <- ci_small[2] - ci_small[1]

# Large sample (n=100)
large_sample <- rnorm(100, mean=100, sd=15)
ci_large <- t.test(large_sample)$conf.int
width_large <- ci_large[2] - ci_large[1]

cat("Small sample (n=10):\n")
cat(" 95% CI:", round(ci_small, 2), "\n")
cat(" Width:", round(width_small, 2), "\n\n")

cat("Large sample (n=100):\n")
cat(" 95% CI:", round(ci_large, 2), "\n")
cat(" Width:", round(width_large, 2), "\n\n")

cat("Larger sample gives narrower (more precise) CI!\n")
cat("Width ratio:", round(width_small/width_large, 2), "\n")
```

6 Practice Problems

6.1 Problem 1: Vaccine Efficacy

A COVID-19 vaccine has 92% efficacy. A clinic vaccinates 50 people.

- What is the probability that at least 45 people are protected?
- What is the expected number protected?
- Create a visualization of this binomial distribution

6.2 Problem 2: Birth Weights

Birth weights are normally distributed with mean 3400g and SD 500g.

- a) What proportion of babies are born with low birth weight ($< 2500\text{g}$)?
- b) What birth weight represents the 10th percentile?
- c) What proportion of babies have birth weights between 3000g and 4000g?

6.3 Problem 3: Sampling Distribution

A population has mean blood glucose of 100 mg/dL with SD 20 mg/dL. You take a sample of 36 patients.

- a) What is the distribution of the sample mean?
- b) What is the probability the sample mean exceeds 105 mg/dL?
- c) Within what range will 95% of sample means fall?

6.4 Problem 4: Confidence Interval

A study measures pain reduction scores for 25 patients: mean reduction = 5.2 points, SD = 2.1 points.

Note: Use the given summary statistics directly. Calculate the CI manually using the formulas from Exercise 4.1.

- a) Calculate a 95% confidence interval for the true mean reduction
- b) Interpret this interval in one complete sentence
- c) Would a 90% CI be wider or narrower? By how much?

6.5 Problem 5: CLT Simulation

Simulate the Central Limit Theorem:

- a) Generate 1000 samples of size 40 from a population with mean=75 and SD=12
- b) Plot the histogram of sample means
- c) Compare the empirical SD of sample means to the theoretical SE
- d) Overlay the theoretical normal curve predicted by CLT

7 Saving Your Work

IMPORTANT: Save your R script file!

```
# At the top of your R script, add:  
# Name: Your Name  
# Date: Today's Date  
# Lab: Week 3 - Probability Distributions and Sampling  
  
# Save your workspace  
save.image("week3_lab.RData")  
  
# To load it later:  
# load("week3_lab.RData")
```

8 AI-Enhanced Learning Tips

Using AI Tools for Distributions

Good prompts for AI assistance:

- “Explain the difference between `dbinom` and `pbinom` with an example”
- “Show me how to calculate a 95% confidence interval in R using t-distribution”
- “Create R code to simulate the Central Limit Theorem with sample size 50”
- “What does this error mean: Error in `qnorm(2)` : argument out of domain $[0, 1]$ ”

AI can help with:

- Explaining which distribution function to use
- Debugging your R code
- Creating visualizations
- Understanding probability concepts
- Suggesting alternative approaches

Remember:

- Always verify AI-generated probabilities
- Test code on simple examples first
- Make sure you understand the concepts, not just the code
- AI is a learning tool, not a replacement for understanding

9 Additional Resources

- R Documentation:
 - `?dbinom` – Binomial distribution help
 - `?pnorm` – Normal distribution help
 - `?t.test` – Confidence intervals and t-tests
- Quick-R: Distribution Functions (<https://www.statmethods.net/advgraphs/probability.html>)
- Textbook: Pagano & Gauvreau, Chapters 7 and 8
- R for Data Science (free online): <https://r4ds.had.co.nz/>

10 Key Takeaways

1. **Binomial:** Use for count data with fixed n , constant p

2. **Normal:** Use for continuous data, foundation for inference
3. **Standard Error:** SD of sample mean = $\sigma/\sqrt{n}$, decreases with larger n
4. **Central Limit Theorem:** Sample means are approximately normal, regardless of population distribution
5. **Confidence Intervals:** Quantify uncertainty, use t-distribution when σ unknown
6. **R Functions:**
 - **d*** functions: density/probability mass
 - **p*** functions: cumulative probability
 - **q*** functions: quantiles (inverse of p)
 - **r*** functions: random generation